

ForesightNav: Anticipatory Reinforcement Learning for UAV Navigation amid Dynamic Obstacles and Large Non-Convex Structures

Abstract—Safe UAV navigation amid dynamic obstacles and large non-convex structures requires anticipating motion conflicts while avoiding local deadlocks induced by challenging geometry. However, existing reinforcement-learning-based policies are commonly trained using instantaneous distance-based safety signals, which can lead to late avoidance maneuvers and myopic decisions. This paper presents ForesightNav, an anticipatory reinforcement-learning framework for UAV navigation built on Proximal Policy Optimization. We introduce a finite-horizon, time-to-collision-aware three-dimensional velocity-obstacle reward that incorporates relative motion and obstacle geometry to encourage proactive avoidance of dynamic obstacles. Moreover, an action-conditioned multi-horizon auxiliary objective provides multi-step near-term outcome supervision by predicting task-relevant rollout outcomes following each observation-action pair. Its losses regularize the observation encoder shared by the actor and critic with semantically separated collision, deadlock, clearance, and progress signals. In purely dynamic-obstacle simulation scenarios, ForesightNav achieves an average navigation success rate of 83.3%, outperforming the NavRL baseline at 70.1% by 13.2 percentage points. In environments containing large non-convex structures, the complete ForesightNav framework reduces deadlocks and achieves a higher post-deadlock escape rate. Zero-shot real-world flights further demonstrate deployment feasibility without fine-tuning. Overall, ForesightNav achieves these improvements without additional online prediction or planning.

I. INTRODUCTION

AUTONOMOUS navigation is essential for unmanned aerial vehicles (UAVs) operating amid dynamic obstacles and large non-convex structures. Beyond reacting to local geometry, a UAV must account for the likely future consequences of its decisions: an action that is collision-free at the current instant may still create a future motion conflict or steer the UAV toward a local deadlock inside geometrically confining structures. Enabling such foresight under limited onboard sensing and computation remains a central challenge in UAV navigation and control.

Classical navigation systems decompose the problem into perception, prediction, planning, and control [1]–[3]. Although interpretable and constraint-aware, these systems rely on accurate models, hand-crafted objectives, and repeated online optimization, which can increase tuning effort and reaction latency. Deep reinforcement learning offers an alternative by learning a direct observation-to-action policy from large-scale simulated experience [4]–[6]. As a representative approach, NavRL learns a direct three-dimensional velocity-control policy with Proximal Policy Optimization (PPO) [6]. Although discounted returns propagate delayed consequences, scalar reward and return signals provide only indirect and entangled supervision of distinct future outcomes related to collision, deadlock, clearance, and progress, potentially limiting proactive behavior in challenging scenarios.

Two recurring failure modes motivate our design. First, instantaneous distance-based safety rewards can assign similar penalties to obstacle configurations with comparable scalar separation but substantially different closing velocities, times to collision, or three-dimensional overlap geometry. A deployment-time velocity-obstacle (VO) shield can filter or correct potentially unsafe actions, but it does not teach the nominal policy to avoid motion conflicts proactively [7]–[9]. Second, locally collision-free actions around large non-convex structures, including U-shaped obstacles, may still steer the UAV toward a local deadlock. Recurrent architectures provide temporal memory but do not by themselves provide explicit supervision of collision, deadlock, clearance, or progress outcomes, whereas model-based methods that rely on online rollouts can increase deployment complexity [10]–[12]. These limitations motivate training-time guidance that combines finite-horizon collision-risk shaping with auxiliary supervision of policy-conditioned multi-step near-term navigation outcomes.

To address these limitations, we present ForesightNav, an anticipatory reinforcement-learning framework that integrates physics-informed reward shaping with task-oriented predictive representation learning. First, ForesightNav incorporates a finite-horizon, time-to-collision-aware reward term derived from a three-dimensional VO formulation into PPO training, encouraging the nominal policy to proactively avoid imminent motion conflicts. Second, we introduce an action-conditioned multi-horizon auxiliary prediction objective that provides multi-step near-term outcome supervision. Conditioned on the current action and the observation encoding shared by the actor and critic, the predictor estimates subsequent rollout outcomes, including future collision and deadlock events, minimum forward clearance over each prediction horizon, and cumulative progress toward the goal. The auxiliary heads provide predictive supervision only during training and are not invoked at deployment. Together, the two signals provide complementary training-time foresight at different learning levels: the TTC-aware 3D-VO reward shapes return-based policy updates using analytic relative-motion risk, whereas the action-conditioned auxiliary losses encourage the shared actor-critic encoder to retain features predictive of realized near-term outcomes following the sampled control; action selection remains governed by PPO.

We evaluate ForesightNav against NavRL, ViGO, and representative learning-based and planning baselines [3, 6] in dynamic-obstacle scenarios, static-dynamic mixed scenarios, and scenarios containing large non-convex structures. ForesightNav and NavRL are evaluated with identical deployment-time VO shield settings. In purely dynamic-obstacle scenarios, ForesightNav achieves an average navigation success rate of

Template figure placeholder

Fig. 1. Overall system architecture of the proposed SwarmNav navigation policy. Raw LiDAR observations and the goal-region amplification input are normalized before being fed into the actor and critic networks, where the critic estimates the state value and the actor outputs the velocity commands.

83.3%, compared with 70.1% for NavRL. In environments containing large non-convex structures, it reduces the deadlock rate by 15.8–25.5 percentage points and achieves a higher post-deadlock escape rate. Real-world UAV flights demonstrate the zero-shot deployment feasibility of the proposed framework.

The main contributions of this work are summarized as follows:

- 1) **Physics-informed 3D dynamic-risk learning:** We introduce a time-to-collision-aware three-dimensional VO reward that provides finite-horizon dynamic-risk supervision and encourages proactive avoidance of dynamic obstacles.
- 2) **Action-conditioned predictive representation learning:** We introduce a multi-horizon auxiliary objective that provides task-oriented supervision for the shared actor-critic encoder during training.
- 3) **Evaluation under dynamic and non-convex geometric challenges:** Experiments demonstrate higher navigation success around dynamic obstacles, fewer deadlocks and improved recovery around large non-convex structures, and zero-shot physical deployment feasibility.

II. RELATED WORK

A. Classical, Learning-Based, and Hybrid Navigation

Classical UAV navigation uses graph search, sampling-based planning, and receding-horizon trajectory optimization. Optimization-based planners such as EGO-Planner, FASTER, and ViGO produce interpretable, constraint-aware trajectories but replan online as maps or predicted obstacle states change [1]–[3]. Learning-based methods shift computation to offline experience: CADRL learns an interaction-aware value function [4], Long *et al.* map range and goal observations

to continuous controls [5], and NavRL uses PPO for direct three-dimensional UAV velocity control and applies a deployment-time shield inspired by the velocity-obstacle (VO) formulation [6]. Although discounted returns propagate future consequences, scalar reward and return signals provide only indirect and entangled supervision of distinct future collision, deadlock, clearance, and progress outcomes.

Hybrid and safety-augmented methods rely on deployment-time planning, recovery, or action projection [13]–[17]. Other approaches use planner demonstrations or privileged expert trajectories during training [18, 19]. Lee *et al.* specifically use privileged time-of-arrival maps to guide quadrotors around large walls and dead ends [20]. ForesightNav retains NavRL’s existing VO shield, but its added VO reward and auxiliary objective are training-only and add no computation to deployment-time action selection. Joint training-time supervision of three-dimensional motion conflicts and semantically separated near-term navigation outcomes remains limited.

B. Velocity-Obstacle-Guided Learning

The VO formulation defines velocities that lead to collision under an assumed relative-motion model [7]. Classical VO methods select velocities outside these forbidden regions; ORCA distributes avoidance responsibility and solves for collision-free velocities online [8]. Used as a shield, however, VO geometry filters a nominal policy command after action generation and therefore does not train proactive avoidance.

VO structure has also shaped reinforcement-learning rewards. DRL-VO uses a two-dimensional directional reward with LiDAR histories and pedestrian kinematics [9], while Han *et al.* include RVO area and expected collision time for differential-drive robots [21]. NavRL instead applies a VO-inspired post-policy shield [6]. ForesightNav derives finite-

horizon, time-to-collision-aware risk from a three-dimensional VO model, including obstacle geometry and vertical separation, to supervise short-horizon policy learning while retaining NavRL's shield as a final runtime filter. Neither the 3D-VO reward nor the VO-inspired shield provides a formal safety guarantee.

C. Predictive Models and Auxiliary Outcome Learning

Predictive navigation methods learn representations or models of future experience. NavRep predicts latent dynamics before policy optimization [10]; RGL combines relational interaction modeling with multi-step crowd lookahead [11]; and BADGR predicts action-conditioned events for online candidate-sequence evaluation [12]. General latent models can introduce prediction error, while online rollout evaluation adds deployment computation. Auxiliary prediction can instead enrich representations while retaining direct action inference: general value functions predict temporally extended signals [22], UNREAL combines reward prediction with auxiliary control and value replay [23], and SPR learns action-conditioned multi-step latent predictions [24].

ForesightNav adopts this auxiliary paradigm to provide multi-step near-term outcome supervision with navigation-specific targets. From each observation-action pair, its heads predict fixed-horizon collision and deadlock events, minimum forward clearance, and cumulative goal progress under the rollout behavior policy. Unlike the PPO critic's scalar expected return, these targets are semantically separated and horizon-specific, explicitly supervising the observation encoder shared by the actor and critic. Together, the auxiliary losses and three-dimensional VO reward provide complementary training-time foresight: the reward shapes short-horizon responses to motion conflicts, while outcome prediction regularizes the shared encoder with near-term collision, deadlock, clearance, and progress information. Neither training objective adds inference-time computation, preserving NavRL's original actor-and-shield inference pipeline.

III. METHODOLOGY

ForesightNav combines finite-horizon dynamic-risk modeling, a partially observable reinforcement-learning formulation, predictive actor-critic representation learning, and an optional deployment-time safety shield. This section presents these components and their integration into policy training and execution.

A. Finite-Horizon TTC-Aware 3D Velocity-Obstacle Modeling

Instantaneous clearance alone cannot distinguish obstacles with similar separation but different relative motion. ForesightNav therefore uses a finite-horizon three-dimensional velocity-obstacle (3D-VO) formulation [7] to quantify dynamic motion conflicts. For a transition from $\mathbf{x}_t$ to $\mathbf{x}_{t+1}$ induced by action $\mathbf{a}_t$, the model is evaluated from the resulting state and realized UAV velocity at $t + 1$.

To represent obstacle geometry without treating all interactions as planar, obstacle i , with size (w_i^x, w_i^y, h_i) , is

approximated by an overlapping vertical stack of spherical primitives. The sphere radius b_i , center spacing l_i , and number of spheres K_i are

$$b_i = \frac{1}{2} \sqrt{(w_i^x)^2 + (w_i^y)^2}, \quad l_i = \max(w_i^x, w_i^y), \quad (1)$$

$$K_i = \left\lceil \frac{h_i}{l_i} \right\rceil.$$

For $j \in \{0, \dots, K_i - 1\}$, the center of primitive (i, j) is

$$\mathbf{p}_{i,j,t}^o = \mathbf{p}_{i,t}^o + \begin{bmatrix} 0 & 0 & \left(j - \frac{K_i - 1}{2}\right) l_i \end{bmatrix}^\top, \quad (2)$$

where $\mathbf{p}_{i,t}^o$ is the obstacle center. The stack is centered vertically on the original obstacle, and all primitives inherit the obstacle velocity $\mathbf{v}_{i,t}^o$.

Before collision evaluation, ForesightNav masks primitives that remain vertically irrelevant over the prediction horizon. For primitive (i, j) , let $r_{i,j,t+1}^z$ and $u_{i,j,t+1}^z$ be its post-transition relative height and vertical relative velocity. The primitive is retained only if the interval bounded by $r_{i,j,t+1}^z$ and $r_{i,j,t+1}^z + H u_{i,j,t+1}^z$ intersects $[-(b_i + m_h), b_i + m_h]$, where $m_h \geq m_z$. This screening interval contains the vertical collision band and therefore does not remove a primitive solely because of a narrower prefilter.

For each retained primitive, define

$$\mathbf{r}_{i,j,t+1} = \mathbf{p}_{i,j,t+1}^o - \mathbf{p}_{t+1}, \quad \mathbf{u}_{i,j,t+1} = \mathbf{v}_{i,t+1}^o - \mathbf{v}_{t+1}. \quad (3)$$

The inflated horizontal and vertical safety scales are $d_{i,j}^{xy} = b_i + m_{xy}$ and $d_{i,j}^z = b_i + m_z$, summarized by $\mathbf{D}_{i,j} = \text{diag}(d_{i,j}^{xy}, d_{i,j}^{xy}, d_{i,j}^z)$. Under constant relative velocity, define the scaled relative trajectory

$$\mathbf{q}_{i,j,t+1}(s, \mathbf{v}) = \mathbf{D}_{i,j}^{-1} [\mathbf{r}_{i,j,t+1} + s(\mathbf{v}_{i,t+1}^o - \mathbf{v})]. \quad (4)$$

The finite-horizon velocity obstacle is then

$$\mathcal{VO}_{i,j,t+1}^H = \{\mathbf{v} \in \mathbb{R}^3 \mid \exists s \in (0, H] : \|\mathbf{q}_{i,j,t+1}(s, \mathbf{v})\|_2 \leq 1\}. \quad (5)$$

ForesightNav evaluates whether the realized velocity $\mathbf{v}_{t+1}$ belongs to $\mathcal{VO}_{i,j,t+1}^H$, rather than explicitly constructing the full set or projecting a new velocity. Define

$$\bar{\mathbf{r}}_{i,j,t+1} = \mathbf{D}_{i,j}^{-1} \mathbf{r}_{i,j,t+1}, \quad \bar{\mathbf{u}}_{i,j,t+1} = \mathbf{D}_{i,j}^{-1} \mathbf{u}_{i,j,t+1}. \quad (6)$$

The boundary-intersection condition is

$$A_{i,j,t+1}s^2 + B_{i,j,t+1}s + C_{i,j,t+1} = 0, \quad (7)$$

where

$$\begin{aligned} A_{i,j,t+1} &= \bar{\mathbf{u}}_{i,j,t+1}^\top \bar{\mathbf{u}}_{i,j,t+1}, \\ B_{i,j,t+1} &= 2\bar{\mathbf{r}}_{i,j,t+1}^\top \bar{\mathbf{u}}_{i,j,t+1}, \\ C_{i,j,t+1} &= \bar{\mathbf{r}}_{i,j,t+1}^\top \bar{\mathbf{r}}_{i,j,t+1} - 1. \end{aligned} \quad (8)$$

For a non-overlapping primitive, a valid time to collision exists when relative motion is nonzero and approaching, the discriminant $\Delta_{i,j,t+1} = B_{i,j,t+1}^2 - 4A_{i,j,t+1}C_{i,j,t+1}$ is nonnegative, and

$$T_{i,j,t+1} = \frac{-B_{i,j,t+1} - \sqrt{\Delta_{i,j,t+1}}}{2A_{i,j,t+1}} \in (0, H]. \quad (9)$$

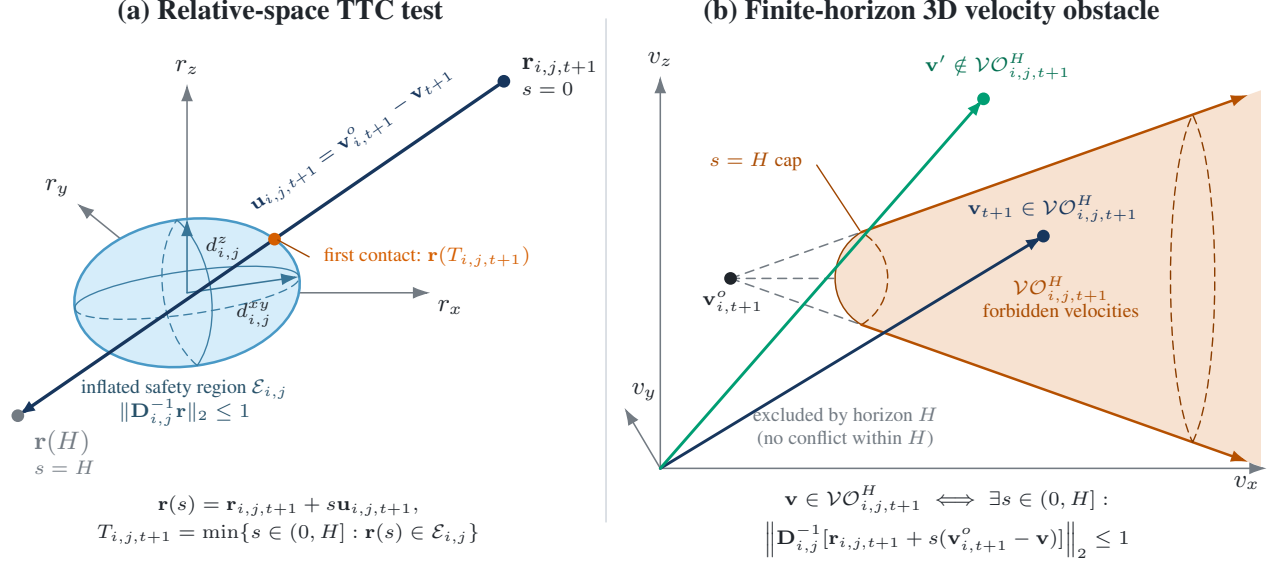


Fig. 2. Geometry of the finite-horizon TTC-aware 3D-VO model for one retained obstacle primitive. (a) The post-transition relative trajectory first intersects the anisotropically inflated safety region at $T_{i,j,t+1}$. (b) The corresponding realized UAV velocity lies inside the finite-horizon velocity obstacle; the $s = H$ cap excludes relative motions that cannot reach the safety region within the prediction horizon. The open end denotes continuation toward larger relative speeds before intersection with the bounded action space. ForesightNav evaluates this membership analytically and does not construct the set or project the policy action during training.

A primitive with $C_{i,j,t+1} \leq 0$ is already inside the inflated region and is assigned unit risk. Otherwise, an in-horizon conflict receives an exponentially decaying TTC-dependent urgency score:

$$\rho_{i,j,t+1} = \begin{cases} 1, & C_{i,j,t+1} \leq 0, \\ \exp(-T_{i,j,t+1}/\tau), & T_{i,j,t+1} \in (0, H], \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

The overall post-transition risk is $\rho_{t+1}^{\text{VO}} = \max_{i,j} \rho_{i,j,t+1}$ over retained primitives. The maximum emphasizes the most imminent conflict, keeps the risk bounded in $[0, 1]$, and prevents its magnitude from scaling with the number of primitives. This training-time measure assumes constant relative velocity over the finite horizon and does not constitute a formal safety guarantee.

B. Reinforcement-Learning Formulation

1) *Problem Formulation*: ForesightNav formulates local UAV navigation as a partially observable Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \Omega, O, \gamma)$. At time t , the policy receives $\mathbf{o}_t \sim O(\cdot | \mathbf{x}_t)$, samples $\mathbf{a}_t \sim \pi_\theta(\cdot | \mathbf{o}_t)$, and induces

$$\mathbf{x}_{t+1} \sim P(\cdot | \mathbf{x}_t, \mathbf{a}_t), \quad r_t = R(\mathbf{x}_t, \mathbf{a}_t, \mathbf{x}_{t+1}). \quad (11)$$

The objective is

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]. \quad (12)$$

Reward and auxiliary targets may depend on latent variables excluded from the policy observation. At the beginning of each episode, a fixed navigation frame G is defined with its origin at the initial UAV position, its horizontal x -axis pointing from

the initial position to the goal, and its z -axis aligned with the world vertical.

2) *State and Observation Space*: Because the complete state is not observable, the policy uses

$$\mathbf{o}_t = (\mathbf{L}_t, \mathbf{s}_t, \mathbf{D}_t), \quad \mathbf{L}_t \in \mathbb{R}^{36 \times 4}, \quad \mathbf{s}_t \in \mathbb{R}^8, \quad \mathbf{D}_t \in \mathbb{R}^{K_{\text{dyn}} \times 10}, \quad (13)$$

where $K_{\text{dyn}} = 5$ is the capacity of the dynamic-obstacle observation. The LiDAR tensor contains 36 horizontal directions at 10° resolution and four elevation angles $\{-10^\circ, 0^\circ, 10^\circ, 20^\circ\}$. Entry $L_t[k, m]$ corresponds to azimuth $\psi_t + 10k^\circ$ and elevation ϕ_m , where ψ_t is the UAV yaw. The scan is indexed in this yaw-aligned sensor frame; roll and pitch are excluded from the ray orientation. A range d is encoded as the clipped proximity $R_L - d$, so larger values indicate closer geometry.

The navigation vector is

$$\mathbf{s}_t = [\hat{\mathbf{r}}_{g,t}^G, d_{g,t}^{xy}, \Delta z_{g,t}, \mathbf{v}_t^G], \quad (14)$$

where $\hat{\mathbf{r}}_{g,t}^G$ is the unit goal direction, $d_{g,t}^{xy}$ is the horizontal goal distance, $\Delta z_{g,t}$ is the vertical goal displacement, and $\mathbf{v}_t^G$ is the UAV velocity. Since G remains fixed, the lateral component of the goal direction retains signed cross-track information after the UAV departs from the initial start-goal line.

For dynamic obstacle i , let $\mathbf{r}_{i,t} = \mathbf{p}_{i,t}^o - \mathbf{p}_t$. Up to K_{dyn} in-range obstacles are ordered by increasing horizontal center distance and represented by

$$\mathbf{d}_{i,t} = [\hat{\mathbf{r}}_{i,t}^G, d_{i,t}^{xy}, \Delta z_{i,t}, \mathbf{v}_{i,t}^{o,G}, q_w(w_i), q_h(h_i)] \in \mathbb{R}^{10}. \quad (15)$$

Here, $\mathbf{v}_{i,t}^{o,G}$ is the obstacle's absolute velocity expressed in G . The width code is $q_w(w_i) = \text{clip}(w_i/\delta_w - 1, 0, 3)$, with $\delta_w = 0.25$ m. Finite obstacles with $h_i \leq 1$ m use $q_h(h_i) = h_i$; taller vertically spanning obstacles use $\Delta z_{i,t} = 0$ and $q_h(h_i) = 0$.

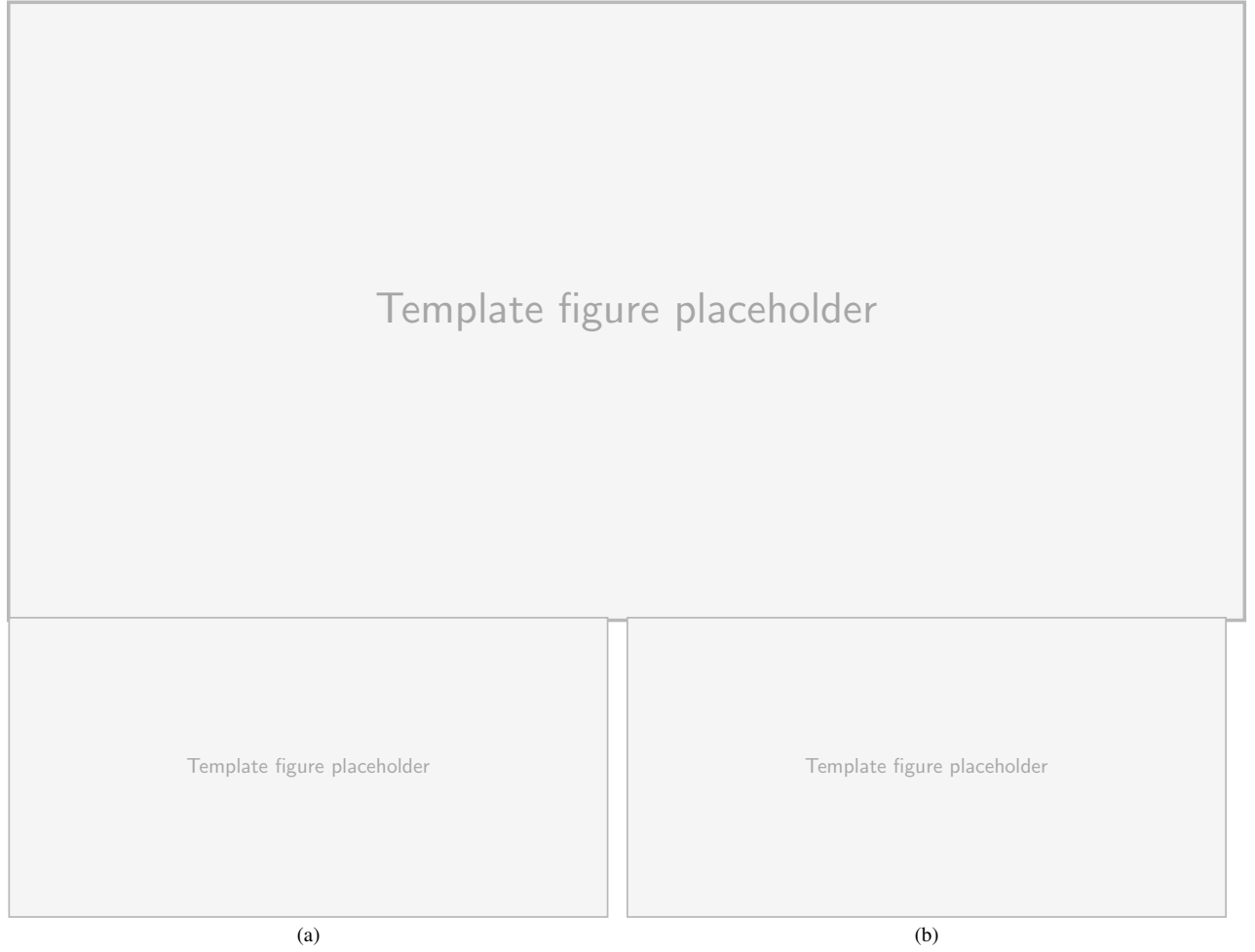


Fig. 3. Geometric illustration of the proposed goal-region amplification in the reward function. (a) Scenario without any obstacles in the reward region. (b) Scenario with obstacles entering the reward region, resulting in the generation of a penalty region.

Missing or out-of-range rows are zero padded. Because the encoder flattens the ordered rows, this distance ordering is part of the observation definition.

3) *Action Space*: The policy action is a bounded three-dimensional velocity command expressed in G :

$$\mathcal{A} = [-v_{\text{lim}}, v_{\text{lim}}]^3. \quad (16)$$

The actor produces positive parameters $\alpha_t, \beta_t \in \mathbb{R}_{>0}^3$ for a factorized Beta distribution. During data collection,

$$\tilde{\mathbf{a}}_t \sim \text{Beta}(\alpha_t, \beta_t), \quad \mathbf{a}_t^G = v_{\text{lim}}(2\tilde{\mathbf{a}}_t - \mathbf{1}), \quad (17)$$

whereas deterministic evaluation uses the distribution mean $\alpha_t/(\alpha_t + \beta_t)$. The affine transformation guarantees that every sampled command satisfies the velocity limit, set to $v_{\text{lim}} = 2.0$ m/s. The command is rotated into the world frame as $\mathbf{a}_t^W = \mathbf{R}_{G \rightarrow W} \mathbf{a}_t^G$ and tracked by a low-level velocity controller. Yaw is regulated separately and is not part of the learned action.

C. Reward Function

The reward combines five principal components that jointly promote anticipatory dynamic-obstacle avoidance,

goal-directed motion, smooth control, static clearance, and altitude regulation:

$$r_t^{\text{shape}} = \lambda_{\text{VO}} r_t^{\text{VO}} + \lambda_g r_t^{\text{goal}} + \lambda_{\text{sm}} r_t^{\text{sm}} + \lambda_{\text{ss}} r_t^{\text{ss}} + \lambda_h r_t^h. \quad (18)$$

This combination distinguishes imminent motion conflicts from geometric proximity and balances safety with efficient, physically regular navigation.

1) *TTC-Aware 3D-VO Risk Reward*: The finite-horizon risk in Sec. III-A enters the reward as

$$r_t^{\text{VO}} = -\eta(n) \rho_{t+1}^{\text{VO}}, \quad \eta(n) = \min\left(\frac{n}{N_{\text{warm}}}, 1\right), \quad (19)$$

where n is the optimization step. The warmup factor gradually introduces anticipatory collision-risk shaping and prevents it from dominating before basic goal-directed behavior is established.

2) *Goal-Progress Reward*: Let $d_t = \|\mathbf{p}_g - \mathbf{p}_t\|_2$. The signed progress over the transition is

$$r_t^{\text{goal}} = d_t - d_{t+1}. \quad (20)$$

Approach to the goal is rewarded, while retreat is penalized with equal magnitude. Consequently, a closed forward-backward cycle cannot obtain positive undiscounted progress reward through asymmetric shaping.

3) *Smoothness Reward*: Abrupt changes in realized velocity are discouraged by

$$r_t^{\text{sm}} = -\|\mathbf{v}_{t+1} - \mathbf{v}_t\|_2. \quad (21)$$

This term favors temporally consistent motion and suppresses oscillatory control during local avoidance.

4) *Static-Safety Reward*: Let $d_{k,t+1}^L$ be the range of LiDAR ray k , N_L the number of rays, and m_s a safety margin. Static proximity is penalized by

$$r_t^{\text{ss}} = -\frac{1}{N_L} \sum_{k=1}^{N_L} \frac{\max(m_s - d_{k,t+1}^L, 0)}{m_s}. \quad (22)$$

Only geometry inside the prescribed margin contributes, while averaging keeps the reward scale independent of LiDAR resolution.

5) *Height Reward*: To discourage unnecessary vertical detours, altitude deviation beyond a tolerance ϵ_z is penalized quadratically:

$$r_t^h = -\max(|z_{t+1} - z_g| - \epsilon_z, 0)^2. \quad (23)$$

The tolerance permits necessary altitude adjustments near obstacles, while the quadratic growth prevents avoidance through excessive ascent or descent.

D. Network Design and Policy Training

ForesightNav organizes policy learning around a shared predictive representation. Modality-specific encoders extract LiDAR and dynamic-obstacle features, which are fused with the navigation state into $\mathbf{z}_t$. The actor generates the bounded Beta policy in Sec. III-B, while the critic estimates the observation-conditioned value $V_\psi(\mathbf{o}_t) = V_\psi(\mathbf{z}_t)$; it does not receive the latent environment state. The actor, critic, and outcome-learning pathway are optimized around this common representation.

To make $\mathbf{z}_t$ informative about the consequences of the selected action, ForesightNav introduces multi-horizon action-conditioned outcome learning. For each horizon $h \in \mathcal{H} = \{1, 3, 5\}$, a lightweight decoder predicts

$$\hat{\mathbf{y}}_t^{(h)} = g_\phi^{(h)}(\mathbf{z}_t, \tilde{\mathbf{a}}_t) = [\hat{c}_t^{(h)}, \hat{b}_{\text{stuck},t}^{(h)}, \hat{f}_t^{(h)}, \hat{p}_t^{(h)}], \quad (24)$$

where $\hat{c}_t^{(h)}$ and $\hat{b}_{\text{stuck},t}^{(h)}$ are collision and persistent-stuck logits, and $\hat{f}_t^{(h)}$ and $\hat{p}_t^{(h)}$ predict minimum forward clearance and cumulative goal progress. Targets are obtained from the collected transition sequence:

$$\begin{aligned} c_t^{(h)} &= \max_{1 \leq k \leq h} c_{t+k}, & b_{\text{stuck},t}^{(h)} &= \max_{1 \leq k \leq h} b_{\text{stuck},t+k}, \\ f_t^{(h)} &= \min_{1 \leq k \leq h} f_{t+k}, & p_t^{(h)} &= \sum_{k=1}^h \Delta d_{t+k}, \end{aligned} \quad (25)$$

where c_t and $b_{\text{stuck},t}$ are the per-step binary collision and persistent-stuck indicators, respectively, and all aggregations are truncated at episode termination. Only the current action is supplied explicitly; later actions follow the rollout policy. Hence, multi-step outputs represent outcomes of the current action under policy continuation rather than under a prescribed action sequence.

For compactness, let $\tilde{f}_t^{(h)} = \text{symlog}(f_t^{(h)})$ and $\tilde{p}_t^{(h)} = \text{symlog}(p_t^{(h)})$, where $\text{symlog}(x) = \text{sign}(x) \log(1 + |x|)$. The predictive objective averages the four tasks over all horizons:

$$\begin{aligned} \mathcal{L}_{\text{MH}} &= \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} [\ell_{\text{BCE}}(\hat{c}_t^{(h)}, c_t^{(h)}) \\ &\quad + \ell_{\text{BCE}}(\hat{b}_{\text{stuck},t}^{(h)}, b_{\text{stuck},t}^{(h)}) \\ &\quad + \ell_{\text{SL1}}(\tilde{f}_t^{(h)}, \tilde{f}_t^{(h)}) \\ &\quad + \ell_{\text{SL1}}(\tilde{p}_t^{(h)}, \tilde{p}_t^{(h)})]. \end{aligned} \quad (26)$$

Policy and outcome learning are integrated through

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + \lambda_{\text{MH}} \mathcal{L}_{\text{MH}}, \quad \lambda_{\text{MH}} = 0.1. \quad (27)$$

The sampled action and rollout targets are treated as fixed inputs. Thus, $\mathcal{L}_{\text{MH}}$ updates the outcome decoder and shared encoders but has no direct gradient path to the actor or critic output heads. PPO makes $\mathbf{z}_t$ useful for action selection and value estimation, while outcome supervision simultaneously makes it predictive of near-term collision, trapping, clearance, and progress.

This predictive actor-critic representation is neither recurrent nor a generative world model: it reconstructs no future observation and performs no latent rollout for action selection. The decoder is omitted during execution, while its representation-level supervision remains embedded in the shared encoder, adding no online inference cost. PPO uses clipped policy and value objectives, generalized advantage estimation, and entropy regularization [25]. A stage-wise curriculum progressively increases dynamic-obstacle density so that increasingly difficult interaction patterns are learned while previously acquired navigation behavior is retained.

E. Deployment-Time Policy Action Safety Shield

The learned policy provides the primary navigation command, but partial observability and unseen interactions can still yield occasional unsafe actions. ForesightNav therefore retains an optional deployment-time shield as a final corrective layer. Given policy command $\mathbf{v}_{\text{RL}}$, the shield passes through an admissible command and otherwise returns a nearby feasible velocity:

$$\mathbf{v}_{\text{cmd}} = \begin{cases} \mathbf{v}_{\text{RL}}, & \mathbf{v}_{\text{RL}} \in \mathcal{V}_{\text{safe}}, \\ \arg \min_{\mathbf{v} \in \mathcal{V}_{\text{safe}}} \|\mathbf{v} - \mathbf{v}_{\text{RL}}\|_2, & \text{otherwise,} \end{cases} \quad (28)$$

where $\mathcal{V}_{\text{safe}}$ is the feasible-velocity set induced by active collision-avoidance and control constraints. The shield is applied after policy inference and does not participate in policy optimization.

The training-time 3D-VO risk and deployment-time shield are complementary. TTC-aware learning encourages the nominal policy to internalize anticipatory avoidance based on obstacle geometry and relative motion. In particular, a rapidly approaching obstacle produces a shorter TTC and a stronger learning signal than a similarly positioned obstacle with weak closing motion. The policy can therefore react before a purely corrective layer has sufficient evidence or time to intervene,

while the shield remains a final safeguard for residual failures rather than the sole source of dynamic-obstacle avoidance.

The evaluation consequently reports ForesightNav both without and with the shield. The former isolates the learned policy’s intrinsic navigation and safety capability, whereas the latter measures the additional robustness produced by correcting residual unsafe commands. This separation prevents improvements from TTC-aware training from being conflated with deployment-time intervention.

IV. EXPERIMENT RESULTS

In this section, we compare the proposed method with several state-of-the-art approaches in simulations across different scenarios and swarm sizes. Furthermore, we also conduct real-world experiments to validate the effectiveness and generalizability of the proposed method.

A. Training Details and Setup

The policy is trained in NVIDIA Isaac Sim using PyTorch. The training scene covers a $40\text{ m} \times 40\text{ m}$ horizontal workspace with geometry represented up to a height of 4.5 m . The maximum UAV speed is 2.0 m/s . To expose the policy to different obstacle scales, the radii of cylindrical dynamic obstacles are set to $\{0.125, 0.25, 0.375, 0.50\}\text{ m}$, while their speeds are sampled from 0.5 to 1.5 m/s . Training is parallelized over 1,024 UAV instances with independently randomized navigation tasks on an NVIDIA GeForce RTX 4090 GPU, and each curriculum stage requires approximately 12 hours. In addition, the hyperparameters used for training are listed in Table I; they were selected and tuned based on preliminary experiments to ensure stable and efficient training.

For quantitative deployment evaluation, we conduct 100 trials for each method in each scenario. A new start–goal pair is sampled for every trial, while obstacles evolve according to the motion process defined by the evaluated scenario. The environment configuration and metric thresholds are held fixed. A trial terminates upon goal arrival, collision, or the 180 s time limit. Let $N_{\text{trial}} = 100$ be the number of trials, and let N_{succ} , N_{coll} , N_{timeout} , N_{dead} , and N_{esc} denote the numbers of successful, collided, timed-out, deadlocked, and post-deadlock successful trials, respectively. We report the following metrics:

- **Success Rate** ($N_{\text{succ}}/N_{\text{trial}}$): The fraction of trials in which the UAV reaches the goal before collision or timeout.
- **Collision Rate** ($N_{\text{coll}}/N_{\text{trial}}$): The fraction of trials terminated by contact with a static or dynamic obstacle.
- **Timeout Rate** ($N_{\text{timeout}}/N_{\text{trial}}$): The fraction of trials that neither reach the goal nor collide within 180 s.
- **Deadlock Rate** ($N_{\text{dead}}/N_{\text{trial}}$): The fraction of trials containing at least one deadlock event. A deadlock is registered when a forward obstacle co-occurs with goal progress no greater than 0.005 m for 40 consecutive evaluation cycles (2 s at 20 Hz).
- **Post-Deadlock Escape Rate** ($N_{\text{esc}}/N_{\text{dead}}$): Among trials in which a deadlock is detected, the fraction that subsequently recover and reach the goal.

TABLE I
KEY HYPERPARAMETERS USED FOR POLICY TRAINING

Parameter	Value	Parameter	Value
Parallel UAVs	1,024	Rollout length	32
PPO epochs	4	Minibatches	16
Discount factor γ	0.99	GAE parameter λ_{GAE}	0.95
Initial LRs (encoder/actor/critic)	$(5, 5, 5) \times 10^{-4}$	Adaptation LRs (encoder/actor/critic)	$10^{-5}, 10^{-5}, 10^{-4}$
Prediction horizons $\mathcal{H}$	$\{1, 3, 5\}$	Outcome-loss weight λ_{MH}	0.1
VO horizon H_{VO}	2.1 s	TTC decay τ	0.75 s
VO reward weight λ_{VO}	1.3	VO warmup	20,000 steps

Together, these metrics distinguish direct navigation success from collision failures, time-limit failures, local trapping, and recovery after trapping.

B. Simulation Results

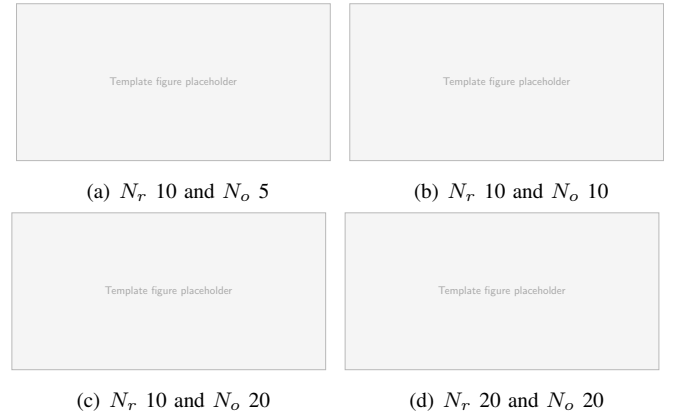


Fig. 4. Trajectories generated by our SwarmNav in four representative scenarios, where all robots successfully reach their goal positions.

In the simulations, we first compare the proposed method with four baseline approaches—RL-RVO [21], PCA, ORCA [8], and RVO [26]—across different circular scenarios and swarm sizes to evaluate its performance. Note that all dynamic obstacles move randomly in an omnidirectional manner. For each condition, 100 independent trials are conducted to collect statistics for the evaluation metrics. The results are reported in Table II, where N_r and N_o denote the number of robots and obstacles, respectively. Moreover, Fig. 4 shows the trajectories of all robots for four representative circular scenarios generated by our method. As shown, our method enables all robots to successfully reach their goal positions across different swarm sizes and dense obstacle environments. From Table II, it can be observed that, compared with the other four baseline methods, our SwarmNav achieves the highest success rate in almost all scenarios. In contrast, the success rates of the four baseline methods vary across different scenarios. The only exception occurs in the absence of dynamic obstacles, where symmetry favors PCA’s diversion strategy; even in this case, SwarmNav still achieves the second-highest success rate and maintains consistently strong performance overall. It should

TABLE II
QUANTITATIVE COMPARISON OVER 100 TRIALS IN FOUR OBSTACLE ENVIRONMENTS WITHIN A $20\text{ m} \times 20\text{ m}$ WORKSPACE.

Environment	Method	Success Rate (%) $\uparrow$	Collision Rate (%) $\downarrow$	Timeout Rate (%) $\downarrow$	Deadlock Rate (%) $\downarrow$	Post-Deadlock Escape Rate (%) $\uparrow$
Static obstacles ($N_{\text{stat}} = 70$)	NavRL	—	—	—	—	—
	ForesightNav	—	—	—	—	—
	ForesightNav w/o 3D-VO	—	—	—	—	—
	EGO-Planner	—	—	—	—	—
Dynamic obstacles ($N_{\text{dyn}} = 60$)	NavRL	—	—	—	—	—
	ForesightNav	—	—	—	—	—
	ForesightNav w/o 3D-VO	—	—	—	—	—
	EGO-Planner	—	—	—	—	—
Mixed static and dynamic obstacles ($N_{\text{stat}} = 20$) ($N_{\text{dyn}} = 40$)	NavRL	—	—	—	—	—
	ForesightNav	—	—	—	—	—
	ForesightNav w/o 3D-VO	—	—	—	—	—
	EGO-Planner	—	—	—	—	—
Static obstacles and large non-convex structures	NavRL	—	—	—	—	—
	ForesightNav	—	—	—	—	—
	ForesightNav w/o 3D-VO	—	—	—	—	—
	EGO-Planner	—	—	—	—	—

also be noted that the success rates of ORCA and RVO drop to zero in the absence of dynamic obstacles due to deadlocks occurring under symmetrical conditions. In terms of average speed and average travel distance, PCA and RL-RVO achieve the best performance in almost all scenarios. This is because these methods do not account for the uncertain motion of dynamic obstacles and thus adopt more aggressive velocities. In contrast, our SwarmNav considers these uncertainties and therefore tends to adopt slightly more cautious speeds toward the goal positions. By comparison, ensuring that all robots safely reach their respective goal positions is the most important metric in swarm robotics navigation. Regarding the average computation time, Table II shows that our proposed method achieves the shortest value in this metric, except for one case, and is followed by RL-RVO. This indicates that reinforcement learning-based approaches significantly outperform traditional methods such as PCA and RVO in computation time for collision avoidance and navigation in robot swarms. In summary, under circular scenarios with dynamic obstacles exhibiting motion uncertainty, the proposed SwarmNav consistently achieves superior performance compared with the four baseline methods, particularly in terms of success rate and computational efficiency. These results demonstrate the effectiveness and robustness of the proposed method.

To evaluate the generalizability of our SwarmNav, we further test large-scale robot swarm navigation in circular scenarios. In this set of simulations, we aim to compare the generalization capability of reinforcement learning-based methods; therefore, we only compare the proposed method with the RL-RVO algorithm [21]. Considering the sensitivity of RL-RVO to dynamic obstacles, obstacles were excluded from this comparison, and the swarm sizes were set to 40 and 60 robots. For each swarm size, 100 independent trials were conducted, and the success rate and average speed are reported in Table III. As shown in Table III, when the swarm

TABLE III
EFFECT OF THE DEPLOYMENT-TIME SAFETY SHIELD OVER 100 TRIALS IN A PURELY DYNAMIC-OBSTACLE ENVIRONMENT WITHIN A $20\text{ m} \times 20\text{ m}$ WORKSPACE.

Method	Shield	Success Rate (%) $\uparrow$	Collision Rate (%) $\downarrow$	Timeout Rate (%) $\downarrow$
NavRL	Off	—	—	—
	On	—	—	—
ForesightNav	Off	—	—	—
	On	—	—	—

size increases beyond 40, despite the space for each robot becoming very crowded, our SwarmNav achieves the best performance in both success rate and average speed. In contrast, when the swarm size reaches 60, RL-RVO succeeds only 7 times out of 100 trials, whereas our method still maintains 58 successful trials. This demonstrates that our SwarmNav possesses superior generalization capability.

C. Real-world Experiments

In the real-world experiments, we aim to further validate the generalization capability of our SwarmNav from simulation to reality. Specifically, we use four Ackermann-model vehicles for testing, each with dimensions of $0.3\text{ m} \times 0.2\text{ m}$, with a maximum linear velocity of 0.6 m/s , a maximum angular velocity of 1.1 rad/s , and a maximum acceleration of 0.5 m/s^2 . Each vehicle is equipped with a Jetson Nano computing platform running Ubuntu 18.04 with ROS Noetic. In addition, each vehicle is equipped with a Mid-360 LiDAR to provide environmental perception data and 10 Hz global localization information obtained using the Point-LIO algorithm [27].

The real-world experiments are conducted in a $5\text{ m} \times 5\text{ m}$ environment, considering four challenging scenarios: a random scenario with static obstacles, a corridor scenario with static

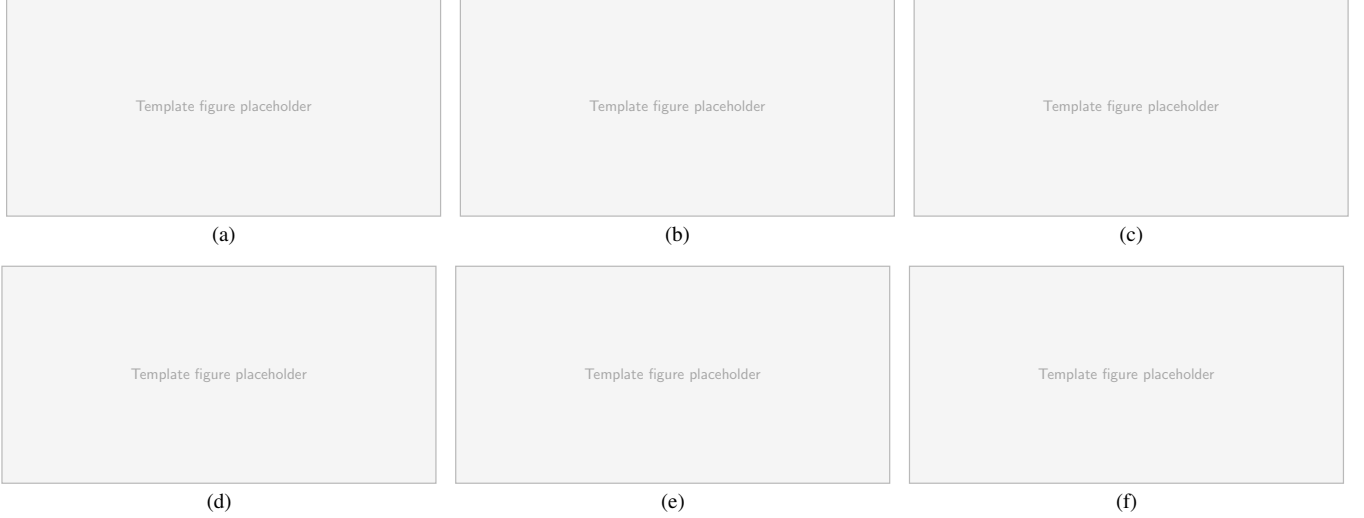


Fig. 5. Real-world experiment results. (a) Random scenario with four static obstacles. (b) Corridor scenario with four static obstacles. (c) Circular scenario with four static obstacles. (d) Circular scenario with two pedestrians. (e) Linear velocity variations in the circular scenario with two pedestrians. (f) Angular velocity variations in the circular scenario with two pedestrians.

obstacles, a circular scenario with static obstacles, and a circular scenario with dynamic obstacles. Note that we simulate dynamic obstacles using pedestrians moving at a constant speed of 0.3 m/s. Finally, the experimental results for the four scenarios are shown in Fig. 5, where Fig. 5(a)–(d) depict the motion trajectories of the vehicles and pedestrians in the four scenarios, and Fig. 5(e) and (f) show the variations of the linear and angular velocities of each vehicle in the circular scenario with dynamic obstacles. As shown in Fig. 5, although our method is trained in a simulation environment and deployed across four real-world scenarios, it is still able to ensure that all robots safely reach their respective destinations. This demonstrates the excellent generalization and robustness of the proposed method. In particular, in the circular scenario with dynamic obstacles, Fig. 5(e) and (f) show that Robots 0 and 3 exhibit noticeable fluctuations in their linear and angular velocities. These fluctuations occur between 3 and 4 seconds after departing toward their destinations, which corresponds to the moment they encounter pedestrians, indicating that each robot proactively adjusts its speed to avoid pedestrians. Overall, the real-world experiments further validate the generalization capability of our SwarmNav and demonstrate its robustness in real-world conditions.

V. CONCLUSION

In this paper, we propose a novel swarm robot navigation method that enables each individual robot to reach its target position quickly and safely in dense and dynamic obstacle environments. We train the collision-avoidance strategy of each robot through a hierarchical reinforcement learning framework, allowing them to handle dynamic obstacles with motion uncertainty. Within this framework, we introduce a novel goal-region amplification strategy and incorporate the RVO formulation into the reward function design. Extensive simulations and real-world experiments demonstrate the effectiveness and strong generalization capability of our SwarmNav. In future work, we plan to explore more complex dynamic scenarios with obstacles of varied shapes.

REFERENCES

- [1] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, “EGO-Planner: An ESDF-free gradient-based local planner for quadrotors,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 478–485, 2021.
- [2] J. Tordesillas, B. T. Lopez, M. Everett, and J. P. How, “FASTER: Fast and safe trajectory planner for navigation in unknown environments,” *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 922–938, 2022.
- [3] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, “ViGO: Vision-aided UAV navigation and dynamic obstacle avoidance using gradient-based B-spline trajectory optimization,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2023, pp. 10 699–10 705.
- [4] Y. F. Chen, M. Liu, M. Everett, and J. P. How, “Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2017, pp. 285–292.
- [5] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, “Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2018, pp. 6252–6259.
- [6] Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada, “NavRL: Learning safe flight in dynamic environments,” *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3668–3675, 2025.
- [7] P. Fiorini and Z. Shiller, “Motion planning in dynamic environments using velocity obstacles,” *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.
- [8] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics Research*, ser. Springer Tracts in Advanced Robotics. Springer, 2011, vol. 70, pp. 3–19.
- [9] Z. Xie and P. M. Dames, “DRL-VO: Learning to navigate through crowded dynamic scenes using velocity obstacles,” *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2700–2719, 2023.
- [10] D. Dugas, J. Nieto, R. Siegwart, and J. J. Chung, “NavRep: Unsupervised representations for reinforcement learning of robot navigation in dynamic human environments,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2021, pp. 7829–7835.
- [11] C. Chen, S. Hu, P. Nikdel, G. Mori, and M. Savva, “Relational graph learning for crowd navigation,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 10 007–10 013, arXiv:1909.13165.
- [12] G. Kahn, P. Abbeel, and S. Levine, “BADGR: An autonomous self-supervised learning-based navigation system,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1312–1319, 2021, arXiv:2002.05700.
- [13] A. Faust, O. Ramirez, M. Fiser, K. Oslund, A. Francis, J. Davidson, and L. Tapia, “PRM-RL: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2018, pp. 5113–5120.

- [14] L. Kästner, X. Zhao, T. Buiyan, J. Li, Z. Shen, J. Lambrecht, and C. Marx, “Connecting deep-reinforcement-learning-based obstacle avoidance with conventional global planners using waypoint generators,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021, pp. 1213–1220.
- [15] S. H. Semnani, H. Liu, M. Everett, A. de Ruiter, and J. P. How, “Multi-agent motion planning for dense and dynamic environments via deep reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3221–3226, 2020, arXiv:2001.06627.
- [16] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, “Recovery RL: Safe reinforcement learning with learned recovery zones,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4915–4922, 2021.
- [17] N. Kochdumper, H. Krasowski, X. Wang, S. Bak, and M. Althoff, “Provably safe reinforcement learning via action projection using reachability analysis and polynomial zonotopes,” *IEEE Open Journal of Control Systems*, vol. 2, pp. 79–92, 2023.
- [18] L. He, N. Aouf, J. F. Whidborne, and B. Song, “Deep reinforcement learning based local planner for UAV obstacle avoidance using demonstration data,” *arXiv preprint arXiv:2008.02521*, 2020.
- [19] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, “Learning high-speed flight in the wild,” *Science Robotics*, vol. 6, no. 59, p. eabg5810, 2021.
- [20] J. Lee, A. Rathod, K. Goel, J. Stecklein, and W. Tabib, “Quadrotor navigation using reinforcement learning with privileged information,” *arXiv preprint arXiv:2509.08177*, 2025.
- [21] R. Han, S. Chen, S. Wang, Z. Zhang, R. Gao, Q. Hao, and J. Pan, “Reinforcement learned distributed multi-robot navigation with reciprocal velocity obstacle shaped rewards,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 5896–5903, 2022.
- [22] R. S. Sutton, J. Modayil, M. Delp, T. Degris, P. M. Pilarski, A. White, and D. Precup, “Horde: A scalable real-time architecture for learning knowledge from unsupervised sensorimotor interaction,” in *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2011, pp. 761–768.
- [23] M. Jaderberg, V. Mnih, W. M. Czarnecki, T. Schaul, J. Z. Leibo, D. Silver, and K. Kavukcuoglu, “Reinforcement learning with unsupervised auxiliary tasks,” in *International Conference on Learning Representations*, 2017, arXiv:1611.05397.
- [24] M. Schwarzer, A. Anand, R. Goel, R. D. Hjelm, A. Courville, and P. Bachman, “Data-efficient reinforcement learning with self-predictive representations,” in *International Conference on Learning Representations*, 2021, arXiv:2007.05929.
- [25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [26] J. van den Berg, M. C. Lin, and D. Manocha, “Reciprocal velocity obstacles for real-time multi-agent navigation,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2008, pp. 1928–1935.
- [27] D. He, W. Xu, N. Chen, F. Kong, C. Yuan, and F. Zhang, “Point-LIO: Robust high-bandwidth light detection and ranging inertial odometry,” *Advanced Intelligent Systems*, vol. 5, no. 7, p. 2200459, 2023.